PART I: Project Documentation & Quick Start (README)

Author: Govind Kumar **Repository:** https://github.com/govind-geethub/lld-practice-platform **Tech Stack:** Go (1.22+), Fiber, SQLite, Tailwind, Mermaid.js

1. Overview

An engineering practice platform designed to help learners iteratively practice Low-Level Design (LLD), submit class hierarchies and architectural notes, receive explainable rubric-driven feedback, and track design evolution across attempts. Built in strict accordance with the **LLD Practice Platform 2-Day Engineering Assignment** and **Candidate Helping Guide**.

2. Key Architectural Features

- **Iterative Practice Loop:** Choose Problem → Design Code & Notes → Submit → Explainable Rubric Feedback → Attempt History → Iterate.
- **Multi-Dimensional Rubric Engine:** 4 structured dimensions (Single Responsibility, Abstraction & Loose Coupling, Design Patterns, Concurrency & State Safety) with explicit Evidence, Concern, Suggestion, and Confidence.
- **Passes Change Tests A & B:** Decoupled `SubmissionFormat` abstraction and pluggable `Evaluator` interface with `CompositeEvaluator` pipeline.
- **Resilient Submission Lifecycle:** Submissions persisted in `EVALUATING` state before evaluation begins with Go context timeout protection.
- **Dynamic UML Diagram Generation:** Parses class and interface hierarchies directly from candidate code to render interactive Mermaid.js diagrams.

3. Quick Start & Execution Guide

```
# 1. Clone repository
git clone https://github.com/govind-geethub/lld-practice-platform.git
cd lld-practice-platform

# 2. Run Go backend server (runs on port 8080)
go run .

# 3. Access in browser
Open http://localhost:8080

# 4. Run automated test suite
go test -v ./...
```

4. Automated Test Verification

All core domain logic, evaluators, pipeline chaining, and edge cases are verified with unit tests:

```
=== RUN   TestDeterministicEvaluator_PassingDesign       --- PASS (0.00s)
=== RUN   TestDeterministicEvaluator_DomainMismatchRejection --- PASS (0.00s)
=== RUN   TestCompositeEvaluator_PipelineChaining        --- PASS (0.00s)
=== RUN   TestDynamicUMLParser                           --- PASS (0.00s)
=== RUN   TestEvaluateDesignLegacyCompatibility          --- PASS (0.00s)
PASS
```

PART II: AI Usage Report (AI_USAGE.md)

Document: Engineering Discernment & AI-Assisted Architectural Decisions

Decision 1: Evaluation Architecture — Multi-Dimensional Rubric vs. Freeform LLM Prompt

AI Suggestion: Delegate evaluation to an unconstrained LLM prompt: "*Evaluate this LLD solution 1-100 and say if it's good.*"

REJECTED **Rationale:** Violates Helping Guide principles. Unconstrained LLM prompts produce inconsistent, ungrounded numbers, hallucinate critiques, and fail during network outages.

ADOPTED ALTERNATIVE Implemented a hybrid pipeline: deterministic validator for domain keywords and god classes, coupled with a 4-dimensional rubric (Criterion → Score → Evidence → Concern → Suggestion → Confidence).

Decision 2: Extensibility Architecture — Pluggable Interface vs. Monolithic Function

AI Suggestion: Write a single monolithic regex validation function inside `main.go`.

REJECTED **Rationale:** Violates **Change Test B**. Tightly couples evaluation rules to HTTP routing, preventing future AI or human evaluators from being added without rewriting practice handlers.

ADOPTED ALTERNATIVE Defined the `Evaluator` interface and `CompositeEvaluator` pattern. New evaluators implement the interface and register in the pipeline with zero changes to HTTP handlers or database persistence.

Decision 3: Resilience & State Lifecycle — Pre-Evaluation Persistence vs. Synchronous POST

AI Suggestion: Evaluate synchronously in the POST handler and return the JSON result immediately without recording state.

REJECTED **Rationale:** If an evaluator times out, crashes, or exhausts API tokens, candidate code is permanently lost. Violates Section 10 Helping Guide requirement.

ADOPTED ALTERNATIVE Explicit state machine: `SUBMITTED → EVALUATING → COMPLETED / FAILED`. Pre-saves to SQLite with status `EVALUATING` before invoking the pipeline under a 10-second Go `context.WithTimeout`.

Decision 4: Dynamic Architecture Diagramming — Code-Extracted UML vs. Static Template

AI Suggestion: Return a static pre-drawn Mermaid diagram of generic Manager-Strategy patterns.

REJECTED **Rationale:** Static diagrams fail to show learners how their specific classes relate or how refactoring changed their architecture.

ADOPTED ALTERNATIVE Implemented `generateDynamicUML()` to parse actual classes, interfaces, and dependencies directly from candidate code and render live Mermaid.js diagrams.

Decision 5: Architecture Scope — Clean Monolith vs. Distributed Microservices

AI Suggestion: Spin up Go API Gateway, Python evaluation worker, Redis queue, and PostgreSQL.

REJECTED **Rationale:** The brief explicitly emphasizes domain modeling over infrastructure complexity. Microservices introduce unnecessary failure modes and deployment friction for a 2-day MVP.

ADOPTED ALTERNATIVE Single modular Go monolith using Fiber and SQLite. Boots in milliseconds with zero dependencies and pristine domain separation.